HARD mode explained

Space Invaders on the DE2-115: how the 21 independent aliens work

1. The idea in one minute

In EASY and MEDIUM the 21 aliens are one rigid block: two timers move the whole group. In **HARD** that block does not exist. Each alien is its own little machine with its own random numbers, its own speed, its own direction and its own timers. They appear one after another at the top, wander sideways at their own pace, and slowly sink. Because nothing is shared, no two aliens move alike and none can drag the others along.

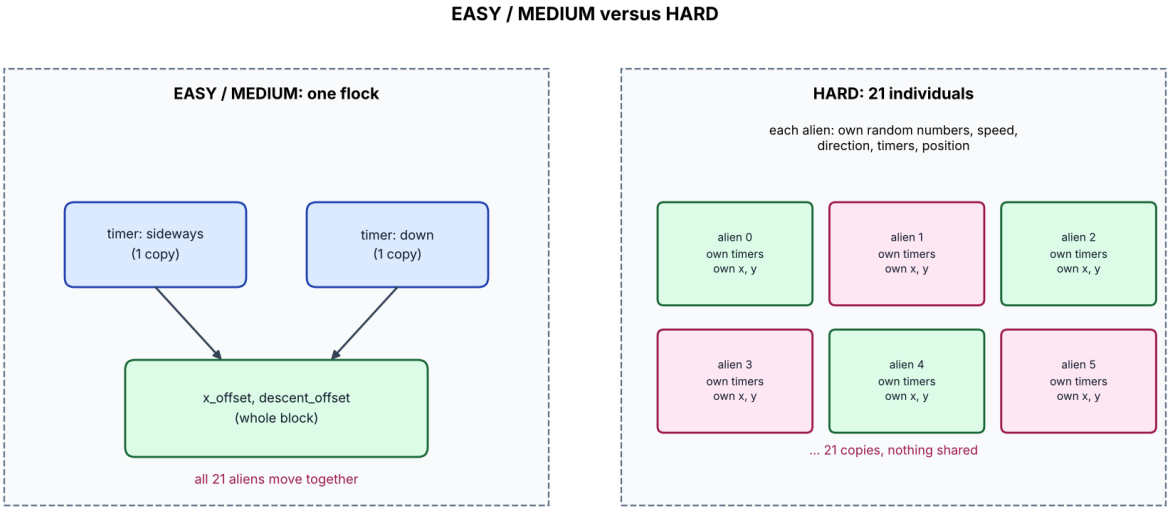


Figure 1. One shared pair of timers (EASY/MEDIUM) versus 21 separate sets (HARD).

In the Verilog this is done with a **generate** loop: the same block of code is copied 21 times, once for each alien number g = 0..20. The copy for alien g is named **gen_alien[g]**, and everything declared inside it belongs to that alien alone.

2. What each alien owns

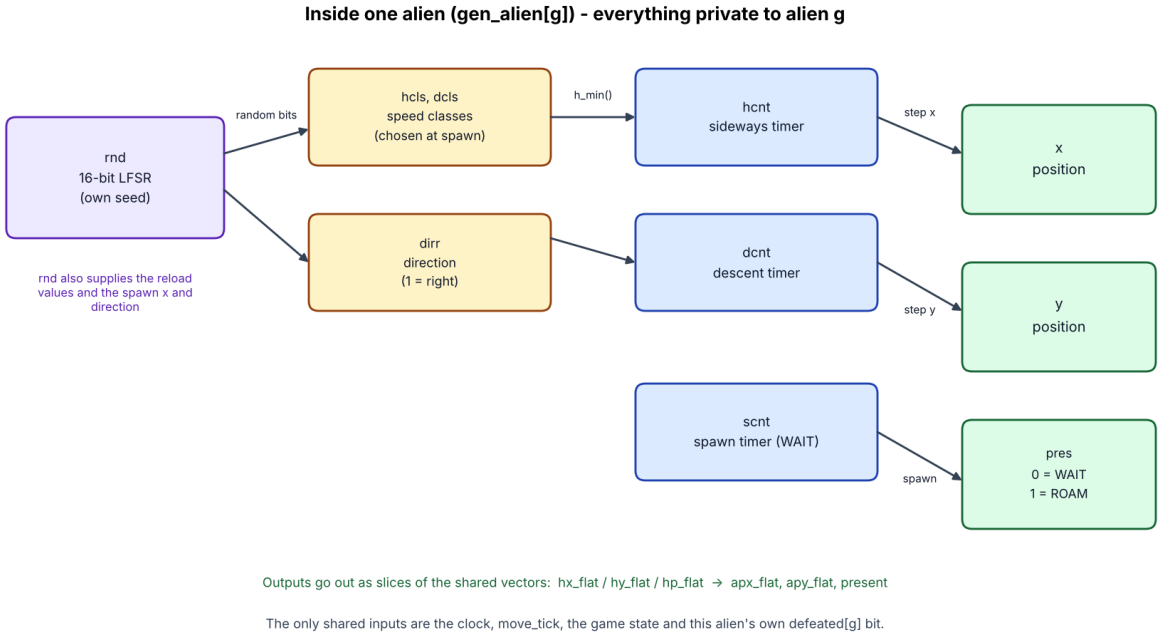


Figure 2. The private registers of one alien and how they connect.

Register	What it holds	Simple meaning
rnd	16-bit LFSR (own seed)	the alien's own random-number generator
pres	0 or 1	the alien's state: 0 = WAIT (not on screen), 1 = ROAM (on screen)

Register	What it holds	Simple meaning
x, y	10 bits each	the alien's position (top-left corner)
dirr	1 bit	direction: 1 = moving right, 0 = moving left
hcls, dcls	2 bits each	sideways speed class and descent speed class (0, 1 or 2)
hcnt	countdown	ticks left before the next sideways step
dcnt	countdown	ticks left before the next downward step
scnt	countdown	ticks left before the alien appears (used in WAIT)

the registers of one alien (src/graphics.v, lines 472-495)

```
genvar g;
generate
    for (g = 0; g < TOTAL_ALIENS; g = g + 1) begin : gen_alien
        localparam integer COL = g % ALIEN_COLS;
        localparam integer ROW = g / ALIEN_COLS;
        localparam [15:0]  SEED = 16'hACE1 ^ (g * 16'h9E37);

        reg [15:0] rnd  = SEED;
        wire       rfb  = rnd[15] ^ rnd[14] ^ rnd[12] ^ rnd[3];
        reg        pres = 1'b0;
        reg [9:0]  x    = 10'd0;
        reg [9:0]  y    = 10'd0;
        reg        dirr = 1'b1;              // 1 = moving right
        reg [1:0]  hcls = 2'd1;              // sideways speed class
        reg [1:0]  dcls = 2'd1;              // descent speed class
        reg [9:0]  hcnt = 10'd0;             // sideways countdown
        reg [9:0]  dcnt = 10'd0;             // descent countdown
        reg [9:0]  scnt = 10'd0;             // spawn countdown (WAIT)

        wire [1:0] hstep = rnd[3]  ? 2'd2 : 2'd1;   // this alien's next sideways step size
        wire [1:0] dstep = rnd[12] ? 2'd2 : 2'd1;   // this alien's next drop size
        wire [9:0] x_fwd = x + hstep;
        wire [9:0] y_fwd = y + dstep;
```

Each alien starts from a *different* random seed (**SEED = 16'hACE1 ^ (g * 16'h9E37)**), so even though all copies run the same code, their random numbers differ and they behave differently.

3. The two states: WAIT and ROAM

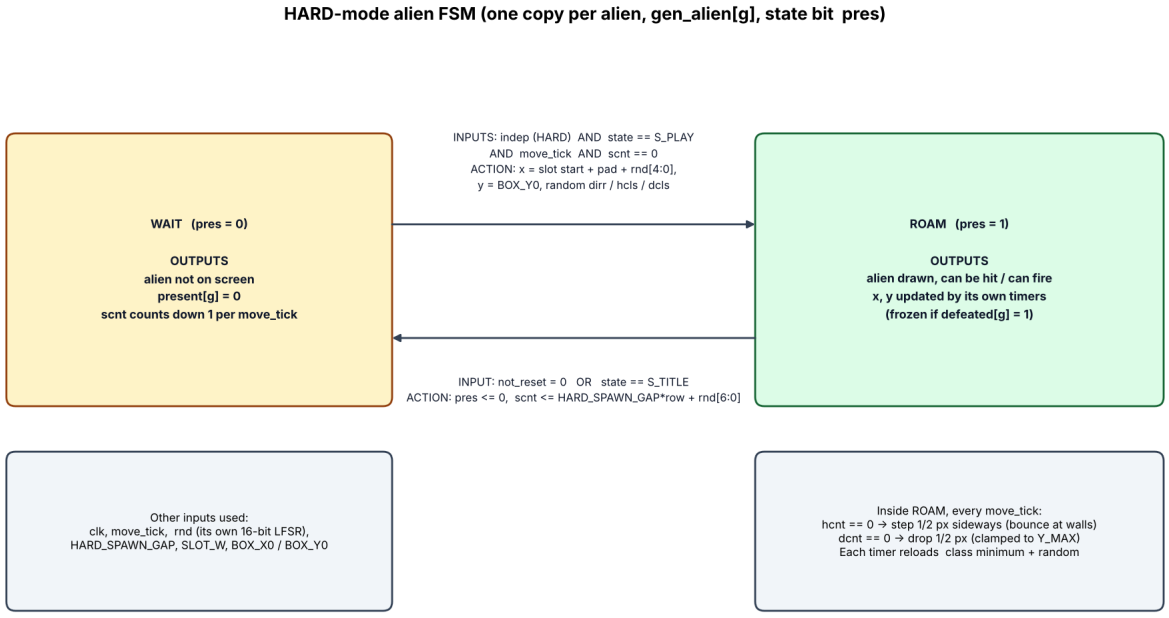


Figure 3. The alien's state machine, with the control inputs on the arrows and the outputs inside the states.

WAIT. The alien is not on screen. This is its state on the title screen and in reset. Its spawn timer **scnt** is loaded with a delay that depends on its row, so the top row appears first. Once the game starts, **scnt** counts down once every 10 ms (**move_tick**); when it reaches 0 the alien appears and the state becomes ROAM.

ROAM. The alien is on screen, can be shot, can fire, and moves by itself. It stays here until the game is reset.

reset / WAIT / spawn (src/graphics.v, lines 499-516)

```
if (!not_reset || state == S_TITLE) begin
    pres <= 1'b0;
    scnt <= HARD_SPAWN_GAP * ROW + rnd[6:0];
end else if (indep && move_tick && state == S_PLAY) begin
    if (!pres) begin
        //---------------- WAIT -> spawn on the top edge
        if (scnt == 10'd0) begin
            pres <= 1'b1;
            x    <= BOX_X0 + COL * SLOT_W + SPAWN_PAD + rnd[4:0];
            y    <= BOX_Y0;
            dirr <= rnd[10];
            hcls <= rnd[6:5];
            dcls <= rnd[9:8];
            hcnt <= {6'd0, rnd[3:0]};
            dcnt <= {6'd0, rnd[15:12]};
        end else begin
            scnt <= scnt - 10'd1;
        end
```

4. Where and when aliens appear

The play area for HARD is a rectangle: x from 16 to 618 and y from 52 (just below the score text) down to 440 (the ship's row). The rectangle is split into 7 vertical **slots**, one per column, each 86 px wide. An alien appears on the **top edge** (y = 52) at a random x inside its own slot, so two aliens do not appear on top of each other.

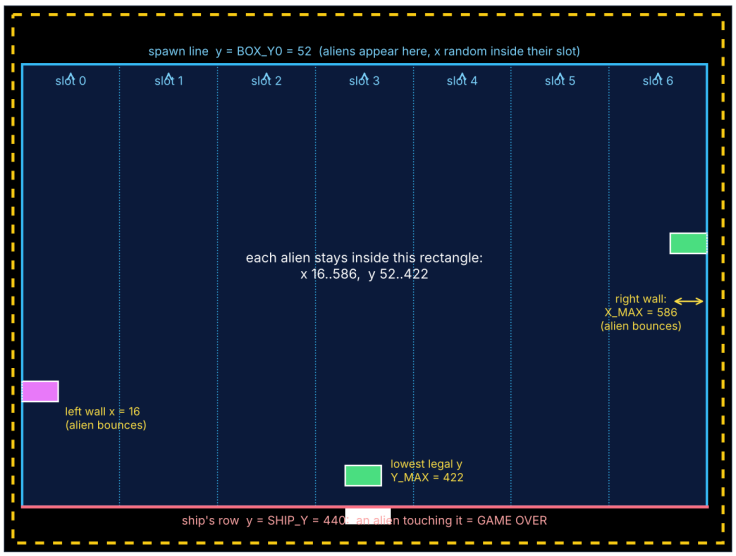


Figure 4. The HARD play rectangle: spawn line, slots, walls, and the ship's row.

Appearance times: row 0 aliens appear within the first 1.3 s, row 1 aliens from 1 s and row 2 aliens from 2 s (each also gets a random extra delay up to 1.3 s). So the wave comes in gradually, not all at once.

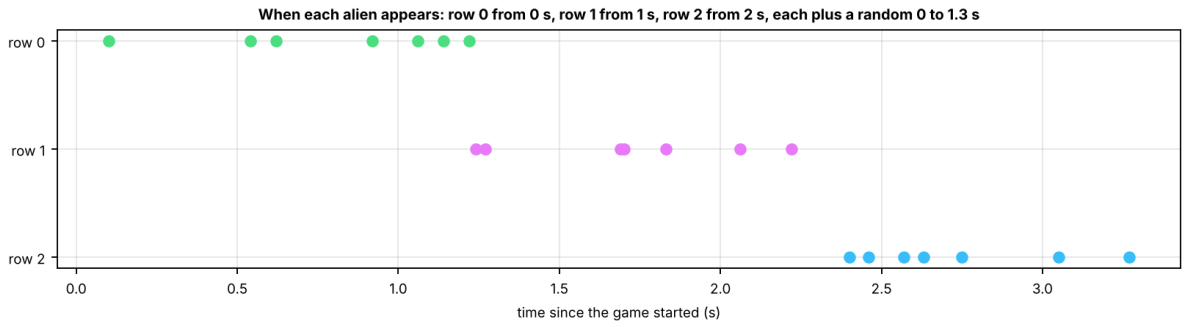


Figure 5. Appearance times of the 21 aliens in one simulated run.

5. How an alien moves

When ROAM, two independent timers run every 10 ms. Think of each timer as an egg timer: when it rings, the alien does one small step, and then the timer is wound up again with a new random wait.

- **Sideways timer (hcnt):** when it reaches 0 the alien moves 1 or 2 pixels in its direction (random size). It bounces off the left wall (x = 16) and the right wall (x = 586), and also reverses at random about once every 64 steps.
- **Descent timer (dcnt):** when it reaches 0 the alien drops 1 or 2 pixels. It is never allowed below y = 422.
- **Reload value:** the next wait is a *minimum for the alien's speed class* plus a random 0..7 (sideways) or 0..15 (down).

ROAM: sideways and downward steps (src/graphics.v, lines 517-540)

```
end else if (!defeated[g]) begin
    //---------------- ROAM: sideways
    if (hcnt == 10'd0) begin
        hcnt <= h_min(hcls) + rnd[2:0];
        if (dirr) begin
            if (x_fwd > X_MAX)        dirr <= 1'b0;      // right wall: bounce
            else if ((rnd[6:4] == 3'd0) && (rnd[9:7] == 3'd0)) dirr <= 1'b0;   // random reversal (1 in 64)
            else                      x <= x_fwd;
        end else begin
            if (x < BOX_X0 + hstep)    dirr <= 1'b1;      // left wall: bounce
            else if ((rnd[6:4] == 3'd0) && (rnd[9:7] == 3'd0)) dirr <= 1'b1;   // random reversal (1 in 64)
            else                      x <= x - hstep;
        end
    end else begin
        hcnt <= hcnt - 10'd1;
    end
    //---------------- ROAM: descent
    if (dcnt == 10'd0) begin
        dcnt <= d_min(dcls) + rnd[11:8];
        y    <= (y_fwd > Y_MAX) ? Y_MAX : y_fwd;
    end else begin
        dcnt <= dcnt - 10'd1;
    end
end
```

Speed classes

When an alien appears it is given two random class numbers, **hcls** and **dcls**, that stay the same for its whole life. The class sets the minimum wait, so it decides how fast that alien is. Fast, medium and slow aliens are mixed in the same game (about 25 % fast, 50 % medium and 25 % slow for each of the two directions).

Class	Sideways minimum wait (HARD_H_MIN)	Sideways speed	Descent minimum wait (HARD_D_MIN)	Descent speed
0 (fast)	3 ticks	about 20 px/s	6 ticks	about 10 px/s
1 (medium; class 3 also)	6 ticks	about 14 px/s	10 ticks	about 8 px/s
2 (slow)	10 ticks	about 10 px/s	15 ticks	about 6 px/s

Speeds are averages: (average step 1.5 px) / ((minimum + average random part + 1) ticks x 10 ms). One tick is 10 ms.

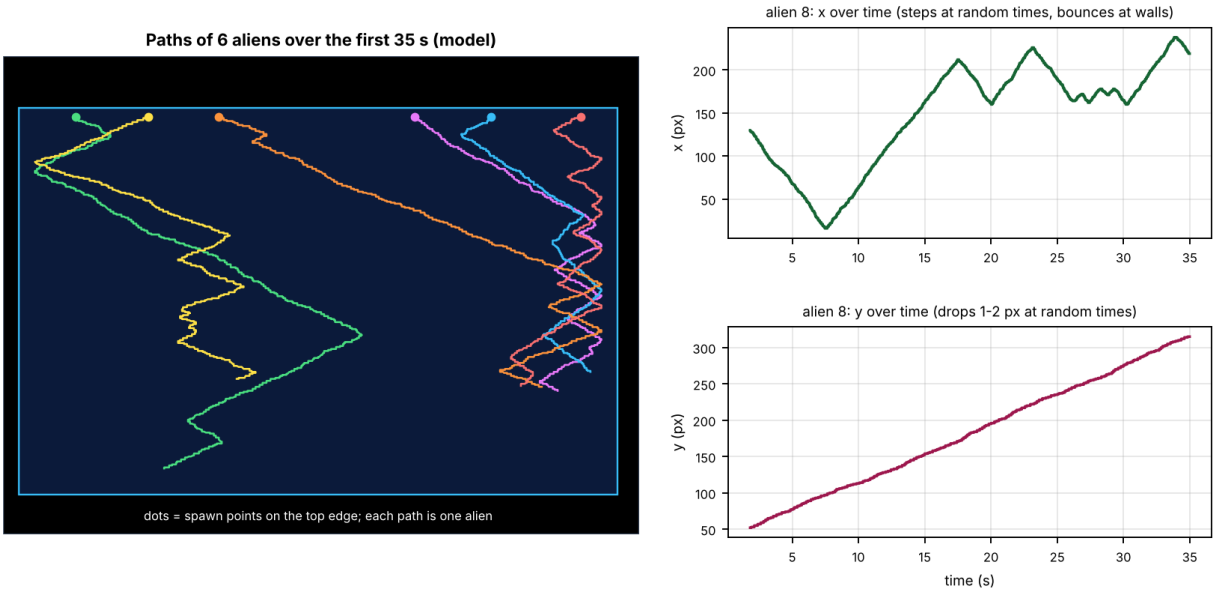


Figure 6. Paths of six aliens over 35 s from the model of the design, and the x and y of one alien against time.

The paths in Figure 6 show the behaviour: each alien drifts one way for a while, bounces off a wall or reverses at random, and slowly sinks. The x graph has slopes of different lengths because the waits between steps are random.

6. Why aliens never leave the rectangle

Every step is checked before it is taken, and the position is clamped. This is why the aliens always stay inside the rectangle, which was verified with the tick-level model over many random start points.

Limit	Value	How it is enforced
Left wall	x >= 16	if the step would go below 16, the direction is flipped instead
Right wall	x <= 586 (X_MAX)	if the step would go past 586, the direction is flipped instead
Top	y >= 52	aliens appear at y = 52 and only ever move down
Bottom	y <= 422 (Y_MAX)	the new y is clamped: y = (y_fwd > Y_MAX) ? Y_MAX : y_fwd

7. How the rest of the game uses the aliens

The aliens' positions leave the generate block through three shared vectors, and the rest of the game reads *only* those vectors. So drawing, shooting and game over work the same whether the aliens move as a block or alone.

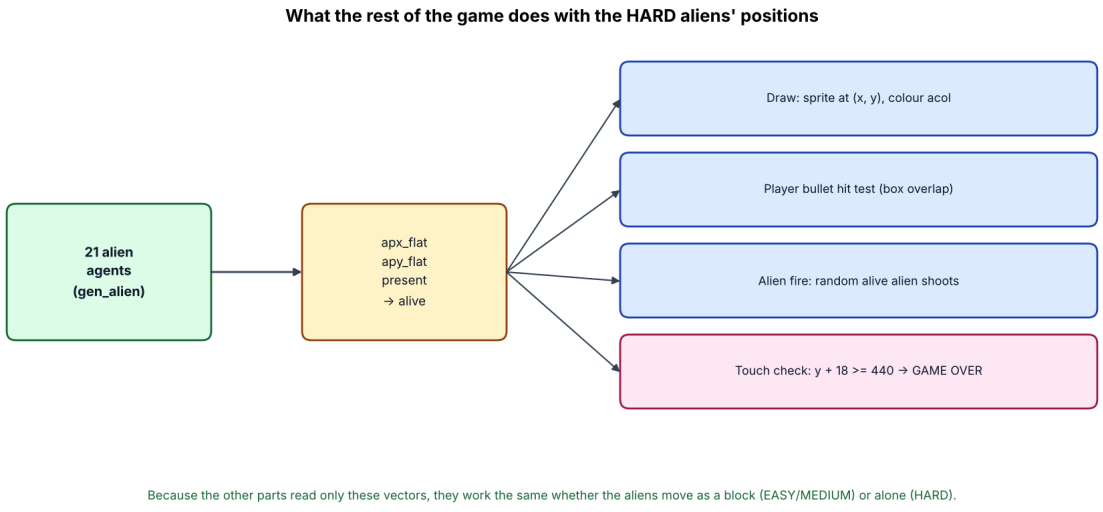


Figure 7. What the rest of the game does with the HARD aliens' positions.

alien positions leaving the generate block (src/graphics.v, lines 438-443, 544-549)

```
wire [10*TOTAL_ALIENS-1:0] hx_flat, hy_flat, apx_flat, apy_flat;
wire [TOTAL_ALIENS-1:0]    hp_flat;
wire [TOTAL_ALIENS-1:0]    present = indep ? hp_flat : {TOTAL_ALIENS{1'b1}};
wire [TOTAL_ALIENS-1:0]    alive   = present & ~defeated;
// y at which the player's bullet gives up (the top of the alien area)
wire [9:0] top_y = indep ? BOX_Y0 : (ALIEN_Y0 + descent_offset);
...
        assign hx_flat[10*g +: 10] = x;
        assign hy_flat[10*g +: 10] = y;
        assign hp_flat[g]          = pres;
        assign apx_flat[10*g +: 10] = indep ? x : (ALIEN_X0 + COL * ALIEN_SPACING + x_offset - MAX_SWEEP);
        assign apy_flat[10*g +: 10] = indep ? y : (ALIEN_Y0 + ROW * ROW_SPACING + descent_offset);
    end
```

- **present:** in HARD it equals the alien's **pres** bit, so an alien that has not appeared yet is invisible, cannot be hit and cannot fire.
- **alive = present and not defeated:** only alive aliens are drawn, can be hit by your bullet and can fire.
- **Your bullet** is tested against each alive alien's *current* box every tick, so it hits aliens where they are now.
- **Alien fire:** a random alive alien shoots from the bottom centre of its current box.
- **Touch check:** if any alive alien reaches the ship's row (y + 18 >= 440), the game is over at once.

8. Facts and consequences

Topic	HARD mode
Alien movement	each alien independent, own speed class and direction
Time to first touch if you never shoot	about 36 s (from the model)
Alien fire wait	HARD_FIRE_MIN = 45, plus a random 0..63 ticks (about 0.8 s)
Alien bullet speed	6 px per tick (600 px/s)
Aliens overlapping	allowed: nothing keeps aliens apart
Bullet hitting two overlapping aliens	only the lower-numbered alien is destroyed
Speeds are fixed at compile time	change HARD_H_MIN0..2, HARD_D_MIN0..2, HARD_FIRE_MIN in graphics.v

Status. This describes the Verilog as written and the checks done with a Python model; the design has not yet been compiled or simulated here, so confirm on the board.

Terms

Term	Meaning
move_tick	One-clock pulse every 10 ms; all motion happens on it.
generate loop	Verilog construct that creates many copies of the same hardware block.
LFSR	Shift register with XOR feedback; used as a random-number source.
slot	The 86 px wide column strip in which an alien of that column spawns.
speed class	A random 0..2 label that sets an alien's minimum wait, hence its speed.
apx_flat / apy_flat / present	Shared vectors carrying all aliens' x, y and on-screen flags.
defeated[g]	1 once alien g has been shot.
X_MAX, Y_MAX	Largest legal alien x (586) and y (422) inside the rectangle.